Day 2 — Java Programming Fundamentals

Topics

 Introduction to Java

 JDK, JRE, and JVM

 Variables and data types

 Operators

 Conditional statements

 Loops

 Input and output

 Basic problem-solving

Tasks

 Install and configure the Java development environment.

 Write programs using variables, operators, conditions, and loops.

 Practice programs such as:

o Check whether a number is even or odd.

o Find the largest of three numbers.

o Calculate factorial.

o Check whether a number is prime.

o Reverse a number.

o Print multiplication tables.

Expected Output

 Java environment configured.

 Basic Java programs completed.

Part 1: Basic Java Interview Questions and Answers

Q1. What is Java?

Answer: Java is a high-level, object-oriented programming language used to develop web applications, desktop applications, mobile applications, and enterprise software.

Example:

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

Output:

```
Hello, Java!
```

Q2. What are the main features of Java?

Answer: The main features of Java are:

1. Platform independent: Java programs can run on different operating systems using the JVM.
2. Object-oriented: Java supports classes and objects.
3. Secure: Java provides features that help develop secure applications.
4. Robust: Java supports exception handling and memory management.
5. Multithreaded: Java supports executing multiple threads.
6. Portable: Java bytecode can run on compatible JVMs.

Q3. What is JDK?

Answer: JDK stands for Java Development Kit. It provides the tools required to develop, compile, and run Java programs.

Q4. What is JRE?

Answer: JRE stands for Java Runtime Environment. It provides the environment and libraries needed to run Java applications.

Q5. What is JVM?

Answer: JVM stands for Java Virtual Machine. It executes Java bytecode and allows Java programs to run on different platforms that support a compatible JVM.

Q6. What is the difference between JDK, JRE, and JVM?

JDK	JRE	JVM
Java Development Kit	Java Runtime Environment	Java Virtual Machine
Used to develop Java programs	Provides the runtime environment	Executes Java bytecode
Includes development tools and runtime components	Includes JVM and runtime libraries	Executes compiled bytecode

Q7. What is a variable in Java?

Answer: A variable is a named memory location used to store data.

Example:

```
int age = 22;
String name = "Suhana";

System.out.println(age);
System.out.println(name);
```

Output:

```
22
Suhana
```

Q8. What are data types in Java?

Answer: Data types specify the kind of value a variable can store.

Java has two main categories:

- Primitive: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`
- Non-primitive: `String`, arrays, classes, interfaces, and others.

Example:

```
int age = 22;
double salary = 25000.50;
char grade = 'A';
boolean isActive = true;
String name = "Suhana";
```

Q9. What are operators in Java?

Answer: Operators are symbols used to perform operations on values and variables.

Operator type	Examples	Purpose
Arithmetic	`+`, `-`, `*`, `/`, `%`	Mathematical calculations
Relational	`==`, `!=`, `>`, `<`, `>=`, `<=`	Compare values
Logical	`&&`, `||`, `!`	Combine

or reverse conditions | | Assignment | =, +=, -= | Assign or update values | | Increment/decrement | ++, -- | Increase or decrease by one |

Example:

```
int a = 10;
int b = 3;

System.out.println(a + b);
System.out.println(a % b);
System.out.println(a > b);
```

Output:

```
13
1
true
```

Q10. What are conditional statements in Java?

Answer: Conditional statements execute different blocks of code depending on whether a condition is true or false.

The main types are:

- `if`
- `if-else`
- `else-if` ladder
- `switch`

Example:

```
int age = 20;

if (age >= 18) {
    System.out.println("Eligible to vote");
} else {
    System.out.println("Not eligible to vote");
}
```

Output:

```
Eligible to vote
```

Q11. What are loops in Java?

Answer: Loops are used to execute a block of code repeatedly.

Java supports:

- `for` loop
- `while` loop
- `do-while` loop

Example:

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Output:

```
1
2
3
4
5
```

Q12. What is the difference between `while` and `do-while`?

Answer:

- A `while` loop checks the condition before executing the body.
- A `do-while` loop executes the body first and checks the condition afterward.

Therefore, a `do-while` loop executes at least once.

Q13. What is input and output in Java?

Answer: Input means receiving data from the user. Output means displaying information to the user.

We commonly use `Scanner` for keyboard input and `System.out.println()` for output.

Example:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Your age is: " + age);
        sc.close();
    }
}
```

Q14. What is basic problem-solving in programming?

Answer: Basic problem-solving means understanding a problem, developing a logical solution, writing code, testing it, and correcting errors.

A common approach is:

1. Understand the problem.
2. Identify the input and expected output.
3. Write the logic or algorithm.
4. Implement the solution in Java.
5. Test the program with different inputs.

Part 2: Java Programming Questions and Answers

Q15. Write a Java program to check whether a number is even or odd.

```
import java.util.Scanner;


public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter a number: ");
      int num = sc.nextInt();


      if (num % 2 == 0) {
         System.out.println("Even number");
      } else {
         System.out.println("Odd number");
      }


      sc.close();
   }
}
```

Enter a number: 7

Odd number

Q16. Write a Java program to find the largest of three numbers.

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter three numbers: ");
      int a = sc.nextInt();
      int b = sc.nextInt();
      int c = sc.nextInt();

      if (a >= b && a >= c) {
         System.out.println("Largest: " + a);
      } else if (b >= a && b >= c) {
         System.out.println("Largest: " + b);
      } else {
         System.out.println("Largest: " + c);
      }

      sc.close();
   }
}
```

Enter three numbers: 10 25 15

Largest: 25

Q17. Write a Java program to calculate the factorial of a number.

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      int n = sc.nextInt();

      if (n < 0) {
         System.out.println("Factorial is not defined for negative numbers");
      } else {
         long fact = 1;

         for (int i = 1; i <= n; i++) {
            fact = fact * i;
         }

         System.out.println("Factorial: " + fact);
      }

      sc.close();
   }
}
```

Output:

```
Enter a number: 5
Factorial: 120
```

Q18. Write a Java program to check whether a number is prime.

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      int n = sc.nextInt();

      boolean isPrime = n >= 2;

      for (int i = 2; i <= n / i && isPrime; i++) {
         if (n % i == 0) {
            isPrime = false;
         }
      }

      if (isPrime) {
         System.out.println("Prime number");
      } else {
         System.out.println("Not a prime number");
      }

      sc.close();
   }
}
```

Output:Enter a number: 7

Prime number

Q19. Write a Java program to reverse a number.

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter a number: ");
      int n = sc.nextInt();

      int reversed = 0;

      while (n != 0) {
         int digit = n % 10;
         reversed = reversed * 10 + digit;
         n = n / 10;
      }

      System.out.println("Reversed number: " + reversed);

      sc.close();
   }
}
```

Output:

```
Enter a number: 1234
Reversed number: 4321
```

Q20. Write a Java program to print the multiplication table of a number.

```
import java.util.Scanner;
```

```
public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter a number: ");
      int n = sc.nextInt();


      for (int i = 1; i <= 10; i++) {
         System.out.println(n + " x " + i + " = " + (n * i));
      }


      sc.close();
   }
}
```

Part 3: Additional Important Interview Questions

Q21. What is the `main()` method in Java?

Answer: The `main()` method is the entry point of a standard Java application. Program execution starts from this method.

```
public static void main(String[] args)
```

- `public`: Accessible to the JVM.
- `static`: Can be called without creating an object.
- `void`: Does not return a value.
- `main`: Method name.
- `String[] args`: Receives command-line arguments.

Q22. What is the difference between `print()` and `println()`?

Answer:

- `System.out.print()` displays text without automatically moving to a new line.

- `System.out.println()` displays text and then moves to a new line.

Q23. What is the modulus operator `%`?

Answer: The modulus operator returns the remainder after division.

Example:

```
System.out.println(10 % 3);
```

Output:

```
1
```

It is commonly used to check whether numbers are even or odd.

Q24. What is the difference between `=` and `==`?

Answer:

- `=` assigns a value to a variable.
- `==` compares primitive values for equality.

Example:

```
int a = 10;

System.out.println(a == 10);
```

Output:

```
true
```

For comparing the contents of Java strings, use `.equals()` rather than `==`.

Q25. What is an infinite loop?

Answer: An infinite loop is a loop that continues indefinitely because its termination condition never becomes false.

Example:

```
while (true) {
    System.out.println("Hello");
}
```

This loop continues until execution is interrupted or the program terminates.